GUI verification

A record of driving the window on the reference export produced by the capture module, of what it shows, and of the defects found while doing so. The window was run in offscreen mode on the same files the demonstration uses:

```
QT_QPA_PLATFORM=offscreen python -m src.ui.main_window \
  --capture tests/corpus/reference_export/corpus_capture_01.normalized.json \
  --rule examples/corpus_rule_v1.json
```

Every statement below was observed on this run; the automated form of the same scenario is `tests/ui/test_e2e.py`.

Verification environment (recorded 2026-10-15, on `main` at `a7c119e`): Linux x86-64, Python 3.13, PySide6 6.8.1, no physical display attached. The primary pass is `QT_QPA_PLATFORM=offscreen`; a second pass (see "Second pass on a real X display" below) ran on a virtual X server (`Xvfb :99`, `openbox`, `QT_QPA_PLATFORM=xcb`) driven with `xdotool`. No pass was run on a physical display, and no human clicked through the window; every step below is automated.

What opens

- Title: `Madrigal protocol laboratory - corpus_capture_01.normalized.json`.
- Menu bar: File (Open demo, Open normalized capture, Open rule, Build rule, Save result, Import PCAP, Quit), Rule (Apply to current direction, Apply to whole capture, Compare versions), Report (Show REPORT.md, Export Markdown, Export HTML), Help (Show start guide, Quick help, About). Every menu and item carries a keyboard mnemonic; Quick help is F1. The File menu keeps the two capture paths distinct: Open normalized capture reads a `*.normalized.json` (or a raw `.pcap`/`.pcapng`, normalized in memory on open), while Import PCAP reads a raw capture and writes its `*.normalized.json` first.
- Central splitter, three columns:
- left: the session tree, group box "Sessions";
- centre: a direction selector, a compact collapsible byte-state legend strip above the hex view, and the hex view, group box "Bytes";
- right: a tab widget with Interpretation, Compare, Report and Version diff.
- Status bar: the byte provenance label on the left, a progress bar on the right (hidden while idle), and a transient message area.
- Interpretation tab, three sub-tabs: Rule (editable text, live parse feedback, field-type completion, and Apply to current direction / Apply to whole capture / Load example buttons), Messages (a table of offset, length, status, fields) and Counterexamples.

The legend is not a text label. Each byte kind is a swatch painted with the same brush the hex view uses, so `gap` and `ambiguity` show their hatch and the names sit next to their colour; the legend cannot drift from the bytes it explains. It is a strip of its own above the hex view, so it cannot overlap the bytes or take width from the ASCII column; its chips fold onto further lines when the window is narrow, and the strip collapses to its title. Each swatch and caption carries an explanatory tooltip, and the whole strip can be hidden from Settings.

The ASCII column shows printable bytes as characters and every byte outside `0x20-0x7e` as a `.`. The dot is a conventional stand-in for a non-printable byte, never a protocol error; the byte's value, print-

able reading and status are always available in the hex column, the byte tooltip and the status line. The column caption is drawn in the same fixed-pitch face and size as the bytes, so it stays aligned with the columns it names.

Session tree (step 1-2)

Opening `corpus_capture_01` lists three sessions, `s1`, `s2`, `s3`, each with its two directions and the byte count per direction. `capture_capture_defects` lists one session whose parent node is marked `[gap, ambiguity]` and whose two direction children are tinted, so a defective stream is visible before it is opened. The status bar reads `3 sessions capture_id sha256:1a2f3637124a diagnostics none` for the clean capture.

Click on a byte and its provenance (step 2)

Selecting session `s1`, direction `A_to_B` and clicking the byte at offset 4 moves the edit cursor onto that byte and writes to the status bar:

```
s1 A_to_B  offset 4   packet 3  seq 1001  ts 1700000000.004
```

The packet index, sequence number and timestamp come from the provenance entry that covers the offset, so a reader can find the byte in the pcap. This is the check the integration checklist asks for, and it passes.

Applying a rule (step 3-4)

With `corpus_rule_v1.json` loaded, applying to `s1 A_to_B` reports `rule v1 matched=60`. Applying to `s2 A_to_B` reports `mismatched=60` and the Messages tab titles become `Messages (60)` and `Counterexamples (60)`. Selecting the first counterexample scrolls the hex view to its offset and the status bar shows that offset again. Every counterexample is tied to real bytes, not to a description of them. A direction with no mismatch is not left blank: the Counterexamples tab reads `no counterexamples in this direction`, so an empty list is never mistaken for a clean result.

Applying to the whole capture reports `matched=60, mismatched=80, incomplete=0, ambiguous=0, uncovered=0, not_applicable=140, outdated=0, unknown=0`, which is the count the research report states. The same run fills the Counterexamples tab from the corpus report, so the tab and the status line agree and each corpus counterexample carries its session and direction; clicking one switches to that session and direction and jumps to its bytes.

`File - Save result` writes the contract result: the file validates against `docs/schemas/result.schema.json` and carries `contract_version`, `rule_id`, `rule_version`, `capture_id`, `messages` and `summary`, so a saved result is the same shape the command line produces.

Version diff (step 5)

After v1 is applied to the whole capture and `corpus_rule_v2.json` is loaded and applied to the whole capture, the Version diff tab shows:

```
report v1 -> v2
  resolved counterexamples: 80
  introduced counterexamples: 0
  unchanged mismatches: 0
  matched delta: +80
  coverage delta: +0.000
```

This is `src.hypothesis.diff.diff_reports` rendered in the window, so the refinement is visible as resolved and introduced counterexamples rather than as a bare change of numbers.

Hypotheses panel (step 4b)

The Hypotheses tab is filled every time a rule is applied, and reads the rule against the messages it applies to. Applying to a single direction scores the candidates over that one stream; applying to the whole capture (Rule, Apply to whole capture) scores them over every scoped stream, which is the comparison the refinement is decided on. With `corpus_rule_v1.json` applied to the whole capture, the tab lists one row per candidate reading of each `hypothesis` field (`command`, `target`, `value`), with the field, the candidate name, support, contradiction count and score. The score is colour-coded: green at or above 0.9, amber between 0.5 and 0.9, grey below. For `command` the top rows are `entropy_enum` and `low_cardinality` at 1.00 (140 support, 0 contradiction), and the row for `constant` (the reading v1 declares) sits at 0.43 (60 support, 80 contradict). Selecting a row emits the first evidence offset and jumps the hex view to that message, the same jump a counterexample click makes. The panel is driven from `src.hypothesis.alternatives.suggest_alternatives`, so it shows the engine's own output, not a second computation. `tests/ui/test_hypotheses_view.py` covers the colour bands, the row-per-candidate layout and the emitted offset, and `tests/ui/test_main_window.py::test_hypotheses_tab_fills_on_apply` checks the tab is populated after an apply.

Defects found during verification

Three defects were found and fixed on this branch; a fourth limit is recorded but not changed.

1. Session diagnostics were never populated. `CaptureModel._read_sessions` built each `SessionInfo` without passing `diagnostics`, so the session tree never showed the `[gap, ambiguity]` marker. Fixed in `src/ui/model.py`.
2. The Version diff panel did not exist: "Compare versions" only posted a status message. Added the tab and wired it to `diff_reports` in `src/ui/main_window.py` and `src/ui/model.py`.
3. Re-applying the same rule marked its own previous run outdated, because `RuleModel.reload` bumped the version on every call. `reload` is now a no-op when the text is unchanged.
4. The window now has a Project menu. A portable project (`src.project`) can be created, opened, extended with the open capture and rule, saved, exported, imported and verified from the menu bar, so scenario step 10 no longer needs the command line. `tests/ui/test_project_management.py` drives the menu and the project format through the real window; the command-line path (`Project.export` / `Project.import_` round trip with matching capture digests) is still covered by `tests/ui/test_e2e.py`.

A later interaction pass over the same window raised further items; the ones closed on this branch are: the corpus run now refreshes the Messages and Counterexamples tabs (they had shown the previous single-direction run); a counterexample can be browsed from the corpus report; `Save result` writes the contract shape above; the legend is painted swatches; the progress bar resets; the empty panels explain themselves; and the file dialogs remember the last directory.

What works

- Opens the reference capture and lists its sessions and directions.
- Renders the stream bytes and highlights diagnostics on the defects file.
- Click on a byte shows its packet index, sequence number and timestamp.
- Applying a rule shows matched and mismatched per direction and over the whole capture, with counterexamples bound to bytes.

- Selecting a counterexample jumps the hex view to its bytes.
- The version diff reports resolved and introduced counterexamples.
- Reopening on the same capture reproduces the same verdicts.
- An empty panel states why it is empty: the Messages tab reads `no messages yet - apply a rule...` before any run, and a clean direction reads `no counterexamples in this direction` rather than showing nothing.
- The file dialogs remember the last directory used, so a second open does not start at the repository root again.
- The progress bar resets to zero and hides when a run finishes, so it never shows a stale fraction of a previous run.
- The rule editor gives live parse feedback (a red status on invalid text, a plain one when the rule parses) and completes field types as they are typed.
- The Project menu creates, opens, saves, extends, exports, imports and verifies a portable project, and names and stores the current rule version, all delegating to `src.project`.

What requires further work

- Response (`B_to_A`) interpretation field by field in the shipped demo rule; the window frames the response stream and the engine honours a `B_to_A` scope, but the demo rule scopes only `A_to_B`.

Traffic column and status line fixes

The live pass above found two places where a value could be read only in part. Both are fixed.

The session tree's `traffic` column held each session's joined direction summary (`A_to_B:300 B B_to_A:420 B`) but was sized to its short header, so the second direction was clipped at the default window size. The column is now sized to the widest summary it actually holds, measured in the font each row is painted with; the header padding was trimmed so the column still fits the default pane without a scroll bar. A panel dragged narrower than the values stops the column at the content width and scrolls instead of clipping, because the floor is re-applied on every resize.

The status line shows the provenance of the selected byte. It now reserves width for its text (bounded, so one very long line cannot push the progress indicator away), carries the full text as a tooltip, and lets the text be selected with the mouse. A window narrow enough to clip the text still shows the whole value on hover.

Both are exercised on a live X display (`docs/screenshots/verify/traffic_and_provenance_fix_live.png`): the byte counts are read in full and the provenance line is not clipped. Three tests cover the behaviour: the column fits every summary, the column keeps its floor on a narrow panel, and the status line reserves room and tooltips its text.

Interface language and theme names

The Russian/English switch was exercised after the live pass. Two items were found and closed:

- The Settings dialog listed the themes by their English names (`Dark`, `Light`) in the Russian catalogue, so the labels did not switch with the language. The theme names are now translated; fixed in `5680af1` (`fix(ui): translate the settings theme names and finish the catalogue`).
- The catalogue had captions that were reachable from the interface but not translated. The catalogue is now complete for the menu, tab and group-box captions, pinned by `tests/ui/test_i18n.py::test_catalogue_covers_the_navigable_captions` and `::test_catalogue_has_no_unfinished_translations`.

The right-hand tab widget switches with the language: on a switch to Russian the first tab reads `Интерпретация` and switches back on return (`tests/ui/test_i18n.py::test_window_retranslates_to_russian`, `::test_window_language_switch_is_reversible`), and an open capture is kept across the switch (`::test_switching_language_keeps_open_capture`).

Second pass on a real X display

The pass above ran `QT_QPA_PLATFORM=offscreen`, which exercises the widgets but never a window manager, a dock, a real font raster or a real input path. A second pass was therefore run on an ordinary X server: `Xvfb :99` at 1680x1050x24 with `openbox` as the window manager and `QT_QPA_PLATFORM=xcb`. Every step of `docs/demo.md` was driven with real pointer and key events (`xdotool`) at the live widget coordinates, and the window was captured after each step. The captures are under `docs/screenshots/verify/`.

What the real display showed, beyond the offscreen pass:

- The window opens and stays open for the whole walkthrough; no step crashes or leaves a stale window behind. Two top-level windows exist (the application window and a hidden 1x1 helper), both owned by the same process.
- The session tree lists all three sessions with their two directions and the byte count per direction. The columns fit their headers.
- Clicking a session fills the hex view; selecting the `A_to_B` direction of `s1` renders its 300 bytes (`B_to_A` renders 420); the direction selector and length label agree.
- The byte-kind fills paint correctly on the live widget: applying v1 to the capture paints the `matched` bytes solid green and the `mismatched` bytes as a hatch, and the legend swatches use the same brushes as the bytes they label.
- The four status fills used for rule outcomes read as distinct on screen; the hatched kinds (`gap`, `ambiguity`, `incomplete`, `mismatched`, `ambiguous`) are a diagonal texture over the surface colour, so a diagnostic range cannot be mistaken for a plain match at a glance.
- The gap and ambiguity diagnostics render on the defects export: `s1 A_to_B` paints 2 bytes as `gap` and 22 as `not_applicable`; `s1 B_to_A` paints 2 as `ambiguity` and 48 as `not_applicable` (`docs/screenshots/verify/clean_dark_gap.png`, `clean_dark_ambiguity.png`).
- Text contrast measured from the rendered pixels is high in both themes: the menu bar is 14.1:1 dark and 16.0:1 light; the hex view is 14.4:1 dark and 17.4:1 light.
- Switching theme does not move or resize anything. The splitter sizes, the tree and hex geometry, the tab panel, the menu and status bars and the two tree column widths are identical before, during and after a dark to light to dark cycle.
- All four Settings tabs (General, Editor, Paths, Advanced) render as distinct pages; each was opened with a real click and captured separately.
- The View and Help menus open and populate over the window (`view_menu_open.png`, `help_menu_open.png`).
- The rule editor parses `corpus_rule_v1.json` and reports `loaded corpus_rule_v1.json rule v1` in its status strip.

Findings from this pass, none of which is a crash:

1. The `traffic` column showed the two directions joined, for example `A_to_B:300 B B_to_A:420 B`. The string needs about 193 px but the column was 131 px at the default window size, so the second direction was clipped: the session column is sized to its header and the traffic column stretches, and the row data was not measured, so a header that fits could still hide the value behind it. This was fixed after the pass (see "Traffic column and status line fixes" above): the column is now sized to the widest summary it holds and keeps that floor on a resize.

2. The offscreen pass reported `3 sessions capture_id ...` in the status bar, while on the live display the same label was the longer `3 sessions capture_id sha2...` after a selection. The label is the same widget; only the visible prefix differs because the label is elided to the left edge of the status bar. Nothing is lost, but the exact string depends on the width, so a reader should not treat it as a fixed value.

Nothing on the live display contradicted the offscreen pass: the counts, the version diff, the hypotheses scores, the byte provenance and the section on `Save result` all reproduce.

Portable project on the command line

The portable project is also verified through `src.project.cli` on the reference capture, which exercises the same format the window's Project menu now drives:

```
python -m src.project.cli pipeline \
  --pcap tests/corpus/corpus_capture_01.pcapng \
  --project /tmp/verifyproj.madrigal --rule examples/corpus_rule_v2.json
```

This writes `manifest.json`, `annotations.sqlite`, `captures/`, `results/` and `reports/`. The digest recorded in the manifest (`sha256:1a2f3637124abcd43049bdf9432ae3d18024276cf7de29efd43562e0c61f272c`) equals a fresh `sha256sum` of the copied capture, the directory is self contained with relative paths, moving it to a different directory and reopening it succeeds, and `export` to an archive followed by `import` into a new target reports no broken files. A project therefore survives being moved between machines.